Journal of the Operational Research Society

ISSN: 0160-5682 (Print) 1476-9360 (Online) Journal homepage: https://www.tandfonline.com/loi/tjor20

Learning automata decision analysis for sensor placement

Tal Ben-Zvi

To cite this article: Tal Ben-Zvi (2018) Learning automata decision analysis for sensor placement, Journal of the Operational Research Society, 69:9, 1396-1405, DOI: 10.1080/01605682.2017.1398205

To link to this article: https://doi.org/10.1080/01605682.2017.1398205

Published online: 19 Dec 2017.

Submit your article to this journal

Article views: 15

View Crossmark data

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tjor20

JOURNAL OF THE OPERATIONAL RESEARCH SOCIETY, 2018
VOL. 69, NO. 9, 1396–1405
https://doi.org/10.1080/01605682.2017.1398205

Learning automata decision analysis for sensor placement

Tal Ben-Zvi

Stevens Institute of Technology, Castle Point on Hudson, Hoboken, NJ, USA

ABSTRACT
This study investigates how to design sensor systems in a way that responds to certain factors in the environment. This decision analysis problem focuses on sensor placement: how to place sensors to find an intruder that is affected by environmental elements. The sensors we use are of two types: the first type detects targets, and the second type detects elements in the environment. Techniques from the learning automata literature are used to develop a detection mechanism. The approach proposed in this study is dynamic, and can adjust to environmental variations. And its rate of detection exceeds static approaches, such as evenly spread sensor configuration. This work has implications for the design of any sensor system in which the physical environment shapes the probability of events occurring.

ARTICLE HISTORY
Received 8 December 2015
Accepted 23 October 2017

KEYWORDS
Decision analysis; learning automata; optimisation; sensor placement

1. Introduction

How can a sensor system be built in such a way that it responds to changes in its environment? First, the environment needs to be sensed. We pick a physical situation that will aid our exploration. For example, periodically, whales enter estuaries and eventually have to be rescued. Imagine sensors in the water which are designed to detect the entry of marine mammals into estuaries. We can imagine other kinds of sensing: detecting human intruders in the same situation.

In considering our problem, we can take a static optimisation approach, and place sensors where we think the whale will be. But how will we know that? Whales only come infrequently into estuaries. In place of looking for an infrequent target, we can look for an easier-to-observe correlate. In this case, we know that the movement of any underwater entity is restricted by the environment. In particular, a whale can only swim against low speed currents. Thus, it is less likely we will find the whale in areas where the current will be against it.

Previous studies mainly concentrated on searching for a certain (moving) target. Those studies used various methods:

(1) Probability – for example, Jiang, Ravindran, and Cho (2013) use probability-based prediction for target tracking in sensor networks; Stone, Streit, Corwin, and Bell (2013) develop a mathematical theory of target tracking from a Bayesian point of view.
(2) Algorithms and heuristics – for example, Bar-Shalom, Willett, and Tian (2011) provide a comprehensive compilation of practical algorithms for surveillance systems operating in a multi-target environment; Benfold and Reid (2011) perform data association to provide guaranteed real-time tracking in high definition video; Breitenstein, Reichlin, Leibe, Koller-Meier, and Van Gool (2011) propose an algorithm for multiperson tracking-by-detection in a particle filtering framework; Lin and Chiu (2005) investigate a near-optimal sensor placement algorithm to achieve complete coverage-discrimination in sensor networks; Tsai, Flagg, Nakazawa, and Rehg (2012) present an off-line algorithm for target segmentation and tracking in video; Ristic, Vo, Clark, and Vo (2011) suggest a quantitative metric for performance evaluation of multi-target tracking algorithms.
(3) Optimisation – for example, Andriyenko, Schindler, and Roth (2012) formulate a target tracking as a discrete-continuous optimisation problem; Martinez and Bullo (2006) study optimal sensor placement and motion coordination strategies for mobile sensor networks; Berclaz, Fleuret, Turetken, and Fua (2011) use K-shortest paths optimisation.

In this context, search theory is a common approach to the problem we study (see, for example, Kwon & Lee, 2011; Vermeulen & van den Brink, 2005). Search theory investigates the selection of an optimal strategy from various potential alternatives or opportunities, under the assumption that delaying this selection is pricy

CONTACT Tal Ben-Zvi tbenzvi@stevens.edu

© Operational Research Society 2017

or detrimental. Therefore, search problems are characterised by optimal stopping rule(s). There are three elements which are present in a search problem when considering a moving target: probability distribution for the target's location and motion, detection function, and constraint on effort (i.e., what resources are available to conduct the search and to what extent) (Haley, 2012).

Another popular technique is the cross-entropy method, a unified approach to combinatorial and multi-extremal optimisation and rare event simulation (Rubinstein & Kroese, 2013). This method has been successfully applied in various application areas, including sensor selection/management (Simonin, Le Cadre, & Dambreville, 2007), where researchers used it to solve the global level when decomposing a hierarchical search problem in two optimisation levels (Le Cadre, 2014).

Here, we look at two types of sensors. The first type is the standard target sensing: we build a sensor that can detect a "moving target" – a whale, or an intruder of any sort. The second is a form of environmental sensing: we build a sensor that detects environmental variations which can change the appearance probability of the target. Then we use the data from the environmental sensors to position the target sensors.

This, then, is our design: use sensors that sense the environment to position target sensors. We can evaluate our design by evaluating detection rates. We can compare our design to other known ways of proceeding, for example, static solutions. But estuary currents continuously change, and therefore, employing such techniques would require pricey re-optimisation and perhaps even sensor replacement. Accordingly, we strive for a dynamic technique where sensors can adjust to the environment. And we want the proposed scheme to achieve close-to-optimal detection rates.

As we will see, we can achieve this: our main contribution is in the design of such a dual-sensor system: environment and target. The system can then be useful in various applied detection problems. By augmenting and applying our preliminary theoretical work in this area (see Ben-Zvi & Nickerson, 2007, 2011), we hope to interest both academicians and practitioners in the potential role sensor systems can play in building adaptable systems.

Now we will proceed with our proposed solution. We will use a mechanism called Learning Automata. This mechanism has a strong theoretical basis and we provide the necessary background next.

2. Learning automata: Background

Our starting point is traditional deterministic control theory and its matching stochastic model in sensor placement. Both theories assume some level of familiarity with the examined system or process (see, for example, Bailey, Shechter, & Schaefer, 2006; Turkogullari, Aras, Altinel, & Ersoy, 2010). That is, the mathematical model, or at least its characteristics, is known (Ben-Zvi & Nickerson, 2007). For example, in studying sensor placement using Markov chains (see, e.g., Liang, Liu, & Carroll, 2010), the problem is modelled by a transition matrix (or a transition graph) containing the (known) transition probabilities: the probability of transitioning from one state to another. However, those theories do not handle well unpredictable changes in the system or the process analysed, as these require setting up analytical expressions for the one-step transition probabilities (and the steady-state probabilities). Such modelling is feasible only for small problems, but not for real world large-sized problems (Gosavi, Das, & Sarkar, 2004). If the problem described here was modelled as Markov chains, the transition matrix would be unknown and too complex to set-up analytically because we assume that changes in the environment occur frequently and at an unknown rate. As a result, further information about the process or the environment is warranted. The area of *Learning automata* is designed to handle this information and incorporate environmental change into the model. We note that to utilise this method, it is not necessary to generate the transition probabilities as in Markov chains; all is required is the knowledge of distribution of the variables that govern the system behaviour, i.e., the feedback from the environment (Gosavi et al., 2004).

An *automaton* is defined as a robot that follows a predetermined series of operations or responds to fixed directions (Thathachar & Sastry, 2004).

The learning component of this process refers to gaining knowledge about the system (or its characteristics) through experience. Learning is taking place by improving performance over time until achieving a mathematically optimum point. This learning is viewed as optimising an unknown function (Zeng & Liu, 2005).

In the past several years, researchers have developed several technique of learning that incorporate environmental data. Those learning techniques, both supervised and unsupervised learning, have been shown to be well suited to be implemented as predictive engines for decision-support for the purpose of pattern recognition, modelling and prediction, and automatic data mapping (Kanevski, Timonin, & Pozdnukhov, 2009).

For example, Broennimann et al. (2012) and Phillips, Anderson, and Schapire (2006) used a learning technique called maximum entropy modelling to estimate the probability of occurrence of a species in contrast to the background environmental conditions. Pyayt, Mokhov, Lang, Krzhizhanovskaya, and Meijer (2011) describe an approach for monitoring flood protections systems based on an artificial neural network learning technique. This technique introduces a set of classification

agents aimed to detect the deviations of the early warning system platform of the *UrbanFlood* European project from its normal state. Li, Heap, Potter, and Daniell (2011) confirmed the effectiveness of random forest, another learning technique, for spatial interpolation of environmental variables using mud content samples in the south-west Australian margin. Jung, Popescu, Kelderman, Solomatine, and Price (2010) applied model trees using environmental data from the Yongdam reservoir in South Korea. Their learning algorithm showed better results for algal growth prediction in the reservoir than using the annual data-set and other algorithms. Landslide susceptibility assessments in different parts of the world have been investigated using various machine learning technique, such as support vector machines, decision trees and logistic regression (e.g., see Lee, 2007; Marjanovic, Kovacevic, Bajat, & Vozenilek, 2011; Tien Bui, Pradhan, Lofman, & Revhaug, 2012). Zhu, Han, and Basar (2010) used no-regret learning for malicious node detection by formulating a game against nature (nature was considered to be an adversary player, picking the worst state). Their analysis generated a mixed saddle-point equilibrium strategy that minimised worst-case losses. For additional learning techniques and applications incorporating environmental data, the reader is referred to Kanevski et al. (2009).

Learning automata models first originated in mathematical psychology. Bush and Mosteller (1955) and Atkinson, Bower, and Crothers (1965) conducted an extensive review of these early studies. Stochastic automata systems are discussed in Varshavski and Vorontsova (1963). The explicit link to control theory was made by Fu (1967) and Fu and Li (1969). The use of networks of learning automata is discussed in Thathachar and Sastry (2004). Obaidat, Papadimitrou, and Pomportsis (2002) and Narendra and Thathachar (2012) review more current theoretical work and applications of learning automata. Gosavi (2009) and Nowe, Verbeeck, and Peeters (2006) link learning automata to the area of reinforcement learning. Reinforcement learning aims to provide a system with the necessary information in order to plan its actions so as to maximise the reward it receives from the environment in which it operates. Learning automata can then be viewed as a stochastic model operating in the framework of reinforcement learning (Nicopolitidis, Papadimitriou, Pomportsis, Sarigiannidis, & Obaidat, 2011). Learning automata can be classified under reinforcement learning in the category of temporal difference (TD) learning methods, a combination of Monte Carlo ideas and dynamic programming ideas (Esnaashari & Meybodi, 2010).

Learning Automata have been applied in several domains, for example, resource allocation (Granmo, Oommen, Myrer, & Olsen, 2007), supply chain optimisation (Wallace, Tsoularis, & Tassabehji, 2006), and wireless networks (Misra, Tiwari, & Obaidat, 2009; Nicopolitidis et al., 2011; and Torkestani & Meybodi, 2010). Timed automata, which refers to clock values that may enable (or disable) transitions and constrain possible behaviours of the automaton, have been used to model and analyse the behaviour of sensor systems; see, for example, Jin, Guan, Wang, and Zeng (2015). For more application areas, see Narendra and Thathachar (2012).

We create the following learning automata framework: we assume that an automaton may carry out a predetermined number of actions in a random environment that comprises numerous possible states. When the automaton selects a certain action, a random response is generated by environment. This response can be either favourable or unfavourable (see Misra, Krishna, & Abraham, 2011; Unsal, Bay, & Kachroo, 1995). This procedure is then repeated, and over time, the system learns what preferred optimal action should be taken in each state. When designing the learning automata we need to consider how actions were taken before and should their corresponded responses channel the course of action. We emphasise that the procedure described above can be executed with not much information on the environment. The positive or negative feedback from the environment is based solely on the actions selected by the automaton. The feedback obtained from the environment may be classified as deterministic or stochastic based on the characteristics of the environment itself.

The learning automata algorithm is executed as follows (see also Gosavi, 2014 and Ben-Zvi & Nickerson, 2007): when the automaton is in a certain possible state, it randomly selects an action and moves to another state. Initially, all actions are equally like to be selected. After the action has been taken, we observe a response or feedback by the environment in the form of a reward or a cost. Then, once we revisit that state, we change the probability of selecting the same action according to the feedback we obtained: a favourable action, that is, if we secured a high reward or a low cost, increases the probability of the action to be taken again. A poor action, i.e., an action that left us with a minimal reward or a high cost, does not significantly change the probability of taking the same action again. The procedure then repeats itself many times.

Listing 1 shows a step-by-step analytical description of the learning automata algorithm. For more details on the algorithm, the reader is referred to Gosavi (2014, pp. 270–274). In contrast to Gosavi's algorithm, we permit more than two actions for the automaton in each state. In addition, we execute the algorithm on multiple automata simultaneously, not only a single automaton.

Listing 1. Gosavi (2014) Learning Automata algorithm – a modified version; see also Ben-Zvi and Nickerson (2007, 2011).

Step 1. Let S denote the set of decision-making states. Also, let A_i denote the set of actions available at state i. Initialise the probability of taking action u_i in state i: $P(i, u_i) = 1/|A_i|$ where $|A_i|$ is the number of possible allowed actions in state i. Set the cumulative reward $C_r(i) = 0$ and the cumulative time $C_t(i) = 0$ for all $i \in S$. Also initialise to zero the total reward earned in the system from the start, $R_{current}$ and the total time clock $T_{current}$.

Step 2. Start the simulation:

If the system's (current) state is $i \in S$, and state i is being visited for the first time, go to *Step 4*; else, in the following order, set:

$R(i) \leftarrow R_{current} - C_r(i)$

$T(i) \leftarrow T_{current} - C_t(i)$,

$\beta(i) \leftarrow R(i)/T(i)$

Step 3. If $u_i = V(i)$ ($V(i)$ is the action taken in the previous visit to state (i) then:

$P(i, u_i) \leftarrow P(i, u_i) + \beta(i)[1 - P(i, u_i)]$.

If $u_i \neq V(i)$ then:

$$P(i, u_i) \leftarrow P(i, u_i) - \frac{\beta(i)[1 - P(i, u_i)]}{|A_i| - 1}$$

Step 4. With probability $P(i, u_i)$, select an action u_i.

Step 5. Set $V(i) \leftarrow u_i$; $C_r(i) \leftarrow R_{current}$ and $C_t(i) \leftarrow T_{current}$

Step 6. Simulate the chosen action. Let the system state at the next decision epoch be j. Also let $g(i, u_i, j)$ be the immediate reward earned in the transition resulting from selecting action u_i.

Step 7. Set $R_{current} \leftarrow R_{current} + g(i, u_i, j)$ and $T_{current} \leftarrow T_{current} + 1$.

Step 8. Set current state i to the new state j; then return to *Step 2* until the probabilities converge.

It is important to determine how to terminate the algorithm. Gosavi et al. (2004) suggest running the algorithm m predetermined number of iterations and then should choose the most probable action for each state. An alternative approach proposed by the authors is to terminate the algorithm when all probabilities converge to either zero or one. This convergence of the algorithm was validated earlier in Wheeler and Narendra (1986). The authors proved that by choosing an appropriate learning rate, the system can converge fast and the result would be arbitrary close to the optimum. The learning rate in machine learning is defined as a step size (in Gosavi's algorithm it is the time units) determining how fast or slow the automaton moves towards the optimum. A very large learning rate will cause the algorithm to diverge; a very small one will slow convergence. Therefore, the learning rate is a decreasing function of time, i.e., an adaptive learning rate (Jacobs, 1988).

Wheeler and Narendra (1986) also showed that it is possible to achieve ε-optimality outcome with respect to different reward criteria by suitably modifying β(i), the feedback from the system (the environment response). An ε-optimality outcome is obtained when calculations are stopped at values that are considered sufficiently close to the optimal solution. It is used in situations where identifying the complete set of solutions is practically unfeasible or computationally too expensive. In the learning automata literature, ε-optimality is the "gold standard" for theoretical convergence (Martin & Tilak, 2012). According to Wheeler and Narendra (1986), regardless of how the feedback from the system (or the environment) is defined, as long as this feedback is normalised, the policy obtained is considered ε-optimal.

3. Problem description

To demonstrate the value of the learning automata algorithm and to empirically show its effectiveness, we examine a real-life problem by expanding a theoretical preliminary work we conducted (Ben-Zvi & Nickerson, 2007, 2011). The problem is to optimise sensor placement that considers information obtained from the environment. More specifically, that information comprises current speed data in the Hudson River (neighbouring New York City) derived from a computation estuarine model used by Stevens Institute of Technology New York Harbour Observing and Prediction System (NYHOPS) described later. Sensor placement research usually does not investigate environmental elements when studying hostile intent or simply just tracking moving objects. We design sensor configurations by incorporating those elements. We look at a one-dimensional view of the Hudson River and assume that the target we are tracking (an intruder, for example) is affected by current speeds (an environmental factor).

Generally, a two-dimensional sensor placement problem, where sensors are aligned together, can be optimised as follows: $P(D_N|x)$ is the probability to detect a certain entity D crossing the Hudson river at a certain point x by employing N sensors. Integrating the total detection probability of the entity attempting to cross the sensor range line over all points across the river yields $P(D_N) = \int_{\text{East bank}}^{\text{West bank}} P(D_N|x)p(x)dx$. Nevertheless, the distribution function that depicts the stream speeds and establishes the probability $p(x)$ is undetermined. In fact, this function may vary over time due to astronomical causes. For example, it is fairly known that in different times during the month currents change their velocity.

The learning automata algorithm we study in this work uses environmental effects, e.g., the current speeds, irrespective to their probability distribution. Consequently, the algorithm serves as a good vehicle to detect moving objects under those conditions.

4. Numerical results

This section presents selected results of the learning automata algorithm. We also compare these results with a typical static sensor placement: evenly distributed stationary sensor network. This assessment proves the effectiveness of the algorithm. First, we use sensors with 99% detection rate that decays exponentially over 110 feet (equivalent to 33.5 m). Each sensor has the following detection function: $0.99\left(1-\frac{d^2}{1122.25}\right)$ if $0 \le d \le 33.5$ and zero otherwise where d represents distance in metres from the sensor. That is, the probability to detect an object at distance d metres from the sensor is as indicated. Note that the detection probability decreases as the object gets farther away from the sensor (for example, when the object is immediately adjacent to the sensor, it will be detected with 99% probability, but if it is 33.5 metres away from the sensor, it will not be detected). We emphasise that the detection capability of a sensor is a parameter in the algorithm and thus, the results of this study are not unique to this function, and other detection functions could produce similar results (see, for example, Ben-Zvi & Nickerson, 2011).

Figure 1 shows the section of the Hudson River we considered. The environmental sensor data we used to measure the variations of the current speeds was obtained from the Pier 88, New York station on 8 July 2015 at 1400 h (2 pm). The data were collected by the Davidson Laboratory at Stevens Institute of Technology through NYHOPS. This maritime system was created at Stevens in 2004 through coordinated efforts from academia, local government, and the United States Navy to permit a real-time assessment of ocean, weather, environmental, and vessel traffic conditions throughout the New York Harbour and New Jersey Coast regions. The system maintains a continuous history of the marine conditions in and around the New York/New Jersey Harbour, and provides a test bed for environmental systems integration into situation awareness and emergency preparedness scenarios, ranging from flooding alerts to search and rescue to chemical spills (Georgas & Blumberg, 2010; Orton, Georgas, Blumberg, & Pullen, 2012). The cross-section we measured is marked in Figure 1 in yellow.

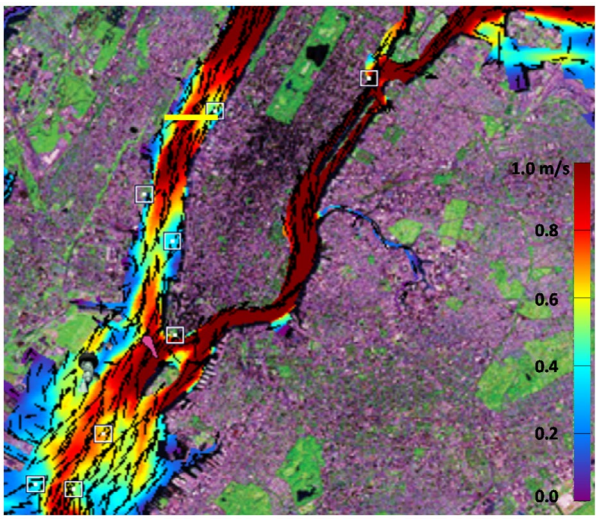

Figure 1. The Hudson River (provided by Davidson Laboratory. Stevens Institute of Technology).

The variations of the current speeds in the cross-section are represented by the curve of Figure 2.

We assume that a moving intruder, whether it is a ship, a boat or even a diver, is more probable to cross the river where the stream is low. Specifically, an attack is more likely to happen near the shores. Figure 3 shows a curve, representing the likelihood of an entity to intersect the sensor range at each point of the river (from the West to the East banks).

The goal is to create a sensor configuration along the cross line between the West and East banks of the river with the aim of improving the detection probability of crossing entities. We first look at a scenario of an evenly distributed set of sensors. Setting the number of sensors to 20, and assuming that the river spans for 1340 m (see also the x axis in Figure 2), the sensors' range of detection runs through the entire span of the river, although the sensors do not overlap. Figure 4 shows how the sensors are arranged. The probability of detecting an object is then 0.6362. We note that based on the detection function, the detection probability differs within the range of each sensor.

Next, starting from the same evenly spread 20 sensors scenario, we executed the learning automata algorithm described above. We ran the algorithm for all sensors simultaneously. The states were the different possible positioning of the sensors while the (permitted) actions were the (possible) movements of each sensor. The reward function was the feedback from the environment (current speeds) translated into the probability density of an intruder, as described

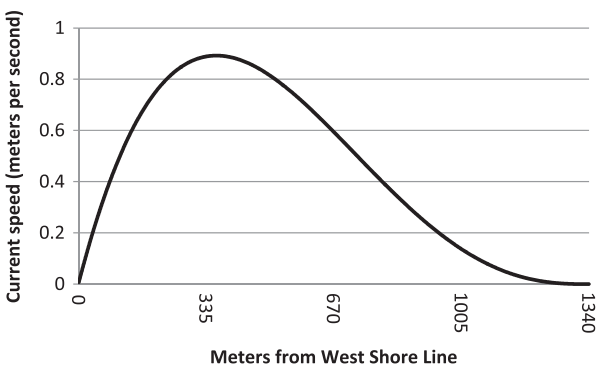

Figure 2. Distribution of current speeds.

above; i.e., the higher the current speed, the lower the probability for an intruder at that (current) state, which is translated to a higher (current) reward. At each time step, the sensors received a feedback (or input), in the form of a reward, from the environment, and then updated their movement probabilities and selected their next state based on those probabilities. The environment, in turn, sent the next feedback to the sensors, and so on. The algorithm converged to the configuration of Figure 5. The overall detection probability was 0.8179, approximately 28% better than the detection probability of the initial evenly spread arrangement. Note that the algorithm instructs not to place any sensors in some sections of the river. Those sections are characterised with extremely high currents and thus, it is very unlikely for objects to choose to cross the river there.

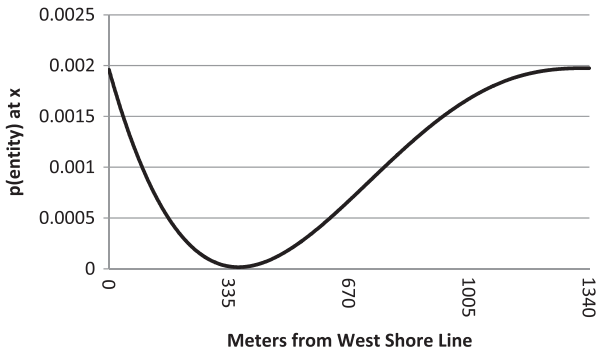

Figure 3. Probability density of an entity (for example, an intruder) intersecting the sensor range at each point of the Hudson river.

We also compared the learning automata results with another method. Our benchmark method is the common notion of evenly spreading the sensors along the river. Each run, we increased the number of sensors by one, starting with a single sensor up to 20 sensors. As can be observed in Figure 6, an even distribution of sensors is strictly inferior to the arrangements generated by the learning automata technique: as we increase the number of sensors, the detection gap between evenly spreading the sensors across the river and the configuration produced by the learning automata technique stabilises at approximately 18 percentage points.

To statistically measure the gap between the two techniques, a confidence interval was constructed to compare detection probabilities produced by both techniques. The result revealed a statistically significant difference at the 95% confidence level. Therefore, we conclude that the learning automata algorithm is robust compared to a static sensor placement.

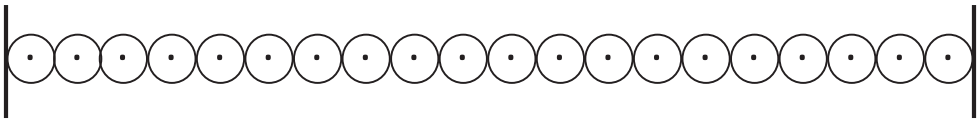

Figure 4. A scenario where 20 sensors are spread evenly across the Hudson River.
Notes: The lines represent the banks of the river (West and East). The dots represent the location of the sensors. The circles signify their associated detection capability.

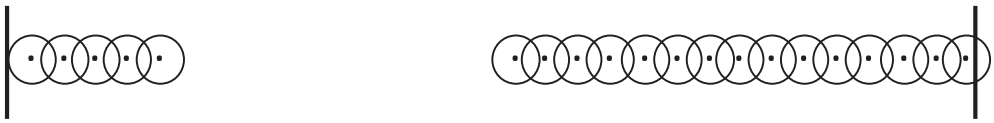

Figure 5. The arrangement generated by the learning automata algorithm.

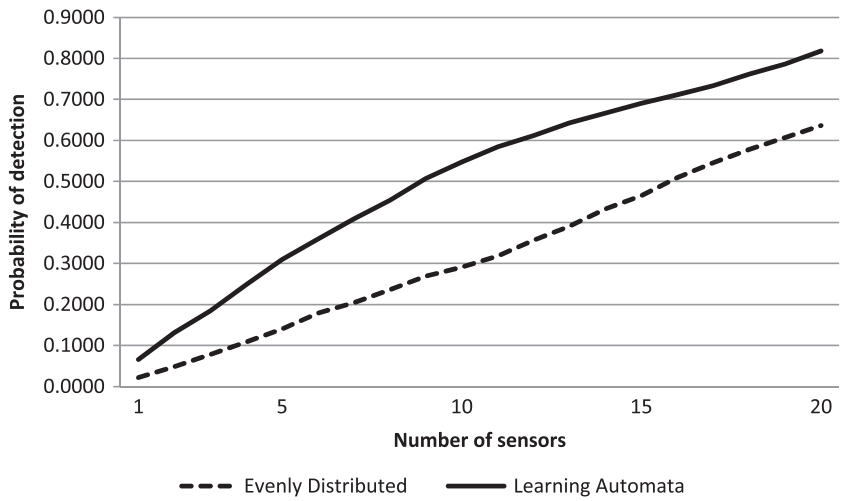

Figure 6. Detection rates of evenly spreading sensors vs. arrangements produced by the learning automata algorithm.

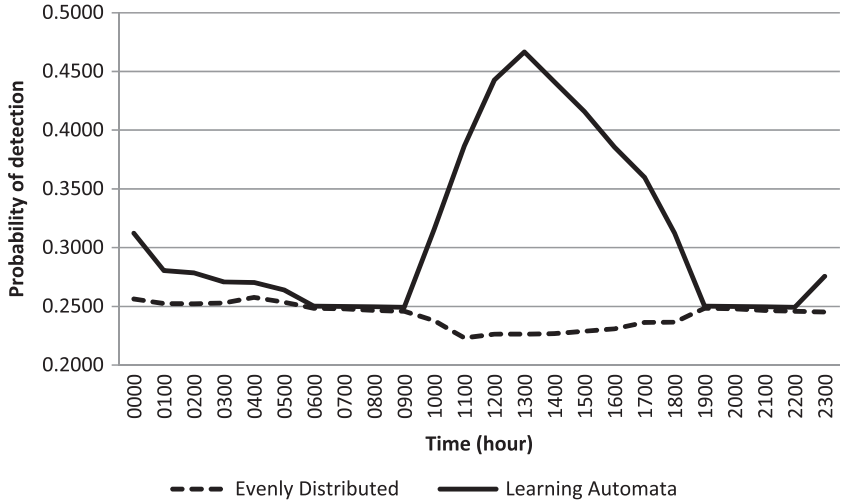

Figure 7. Detection rates of evenly spreading sensors vs. arrangements produced by the learning automata algorithm over a 24 h period on 8 July 2015.

Next, we compared the two techniques over a 24-h period. We used data from the same maritime station (Pier 88, NY) on 8 July 2015. For this experiment, we used different sensors with the following detection function: $0.99\left(1-\frac{d^2}{625}\right)$ if $0 \le d \le 25$. Figure 7 shows the detection probability for both evenly spread sensors and sensors placed by the learning automata algorithm. For both techniques, 10 sensors were employed. The calculations show that the learning automata algorithm always exhibits superior results to the evenly spread configuration. Moreover, the algorithm works significantly better when current speed varies across the cross section (in this case, between 0900 h and 1900 h).

5. Extension: the cost of sensor deployment

Although the thrust of this study is to design sensor systems that adapt to changes in the environment, security officers may also weigh financial considerations, namely, the cost of sensor deployment. Herein, we assume that the security officer is optimising a utility function that evaluates both detection rates and the cost of deploying the sensor network. Deploying additional sensors will probably not decrease the detection probability, and most possibly increase it; yet, this (higher) detection rate comes at a price – the cost of deploying the additional sensors. Consequently, the officer shall consider the level of detection on the one hand and the cost of the sensors on the other.

To demonstrate how to incorporate the cost of sensor deployment into the model, we refer to the numerical data of Section 4 and provide in Table 1 information on the number of sensors, and their associated detection rates and utility. The nature of the security officer's utility function depends on his or her preferences. One example of a utility function is a function based on the Expectation criterion. Another example of a utility function is a satisficing function, in which the objective is to minimise the cost of achieving a certain detection probability. This criterion was first suggested by Simon (1959) and is commonly used in the literature (Chernonog, Avinadav, & Ben-Zvi, 2015). Here we employ a simple linear function which optimises the expected utility, considering both detection rates and sensor cost: $100P(D_N)$-3N (where the coefficient "3" represents the cost of each sensor). We leave the analysis and discussion of different utility functions and function parameters to future research. We note that the number of sensors and the detection rate columns in Table 1 correspond with the data illustrated in Figure 6.

The information in Table 1 shows that as the number of sensors increases (increasing the cost of network deployment), the detection rate increases as well; although, not linearly. Based on the utility information in the table, the security officer would choose to deploy 11 sensors and will achieve a detection probability of 0.5843.

Table 1. Number of sensors, detection rates, and associated utility.

Number of sensors	Detection rate	Utility
1	0.0657	3.5696
2	0.1309	7.0940
3	0.1841	9.4143
4	0.2481	12.8128
5	0.3099	15.9867
6	0.3600	17.9987
7	0.4088	19.8797
8	0.4545	21.4489
9	0.5077	23.7654
10	0.5477	24.7733
11	0.5843	25.4250
12	0.6117	25.1750
13	0.6426	25.2593
14	0.6669	24.6865
15	0.6903	24.0253
16	0.7113	23.1250
17	0.7333	22.3339
18	0.7620	22.2047
19	0.7867	21.6671
20	0.8179	21.7924

6. Discussion

When examining security systems, the typical assumption is that intruders are equally likely from all (possible) directions. That usually leads to equi-space sensors. However, environmental elements could have an impact, and by incorporating these effects, we can often better detect intrusions. The results show that in environments that change over time, employing a learning algorithm with environmental sensing is superior. This type of sensing does not require computing any future projections on the environment, or even employing extensive optimisation techniques. The learning algorithm suggests instead moving the sensors in reaction to the changes in the environment. And by doing so, they find their optimal detection positions.

The solution we propose in this work exhibits an interesting framework from the standpoint of situation management: sensors can sense certain elements in the environment that affect the positioning of other sensors that search for a certain target or an intruder. Clearly, correct detection depends on the quality of the detection mechanism, which includes obtaining accurate environmental data. We leave the study of accurate detection, and the impact of false positives and false negatives to future research.

This concept of dual-sensing – one for the context, and the other one for certain infrequent occurrences – exhibits other possible applications. For example, fairly similar to the river current sensor detectors, air contamination sensors have the ability to monitor air flows. This might assist in focusing on the vicinity where pollution is most likely to transpire. By monitoring currents, measuring air flows or differences in air pressure, this type of systems examines physical realisms that also impact the probabilities of certain events occurring. Our conjecture is that dual-sensing systems that combine together environmental and detection sensing, will have broad applicability, and may facilitate the design of more robust systems. This conjecture can be tested first through simulation using as the environment easily obtained datasets, such as weather sensor data. Then it can be tested through deployment. This study presented how this arrangement can become useful in detecting intruders.

7. Conclusions

We all wish our systems were more adaptable to changing environments. This study shows that a dual-sensor system can be useful in detecting infrequent events. Information obtained from sensing the constant changes in the environment can be used to position sensors in a way that is more likely to detect certain targets. We take this approach by applying learning automata theory. The examined case showed that our approach is superior to the static technique, and has the advantage of being driven effects from the environment.

Our key contributions in this paper are as follows: (1) we extend Gosavi's learning automata algorithm (2014) and allow more than two actions for the automaton in each state of the system; (2) we execute the learning automata algorithm on multiple automata simultaneously, not merely on a single automaton; and (3) we employ the model in a security setting, using real environmental data to optimise the placement of maritime sensor systems while considering sensor cost.

The approach described in this study could also be advantageous in protecting against asymmetric attacks, which include the use of surprise in its operational and strategic aspects, or the use of terrorism by proxy. This type of attacks is becoming more common in recent years and is dreadful if not detected. Taking action, by observing the environment and monitoring changes, can significantly improve the odds of identifying such an attack.

Disclosure statement

No potential conflict of interest was reported by the author.

References

Andriyenko, A., Schindler, K., & Roth, S. (2012). Discrete-continuous optimization for multi-target tracking. *Computer Vision and Pattern Recognition (CVPR)*IEEE Conference on, 1926–1933.

Atkinson, R. C., Bower, G. H., & Crothers, E. J. (1965). *An introduction to mathematical learning theory*. New York, NY: Wiley.

Bailey, M. D., Shechter, S. M., & Schaefer, A. J. (2006). SPAR: Stochastic programming with adversarial recourse. *Operations Research Letters, 34*(3), 307–315.

Bar-Shalom, Y., Willett, P. K., & Tian, X. (2011). *Tracking and data fusion*. Storrs, CT: YBS publishing.

Benfold, B., & Reid, I. (2011). Stable multi-target tracking in real-time surveillance video. *Computer Vision and Pattern Recognition (CVPR),* 3457–3464.

Ben-Zvi, T., & Nickerson, J. V. (2007). Responding to changing situations: Learning automata for sensor placement. *Proceeding of the Military Communications Conference (MILCOM)*, Orlando, Florida.

Ben-Zvi, T., & Nickerson, J. V. (2011). Decision analysis: Environmental learning automata for sensor placement. *IEEE Sensors Journal, 11*(5), 1206–1207.

Berclaz, J., Fleuret, F., Turetken, E., & Fua, P. (2011). Multiple object tracking using k-shortest paths optimization. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 33*(9), 1806–1819.

Breitenstein, M. D., Reichlin, F., Leibe, B., Koller-Meier, E., & Van Gool, L. (2011). Online multiperson tracking-by-detection from a single, uncalibrated camera. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 33*(9), 1820–1833.

Broennimann, O., Fitzpatrick, M. C., Pearman, P. B., Petitpierre, B., Pellissier, L., Yoccoz, N. G., … Graham, C. H. (2012). Measuring ecological niche overlap from occurrence & spatial environmental data. *Global Ecology and Biogeography, 21*, 481–497.

Bush, R. R., & Mosteller, F. (1955). *Stochastic models for learning*. New York, NY: Wiley.

Chernonog, T., Avinadav, T., & Ben-Zvi, T. (2015). Pricing and sales-effort investment under bi-criteria in a supply chain of virtual products involving risk. *European Journal of Operational Research, 246*(2), 471–475.

Esnaashari, M., & Meybodi, M. R. (2010). A learning automata based scheduling solution to the dynamic point coverage problem in wireless sensor networks. *Computer Networks, 54*(14), 2410–2438.

Fu, K. S. (1967). Stochastic automata as models of learning systems. In J. T. Lou (Ed.), *Computer and information sciences II* (pp. 393−431). New York, NY: Academic Press.

Fu, K. S., & Li, T. J. (1969). Formulation of learning automata and automata games. *Information Science, 1*(3), 237–256.

Georgas, N., & Blumberg, A. F. (2010). Establishing confidence in marine forecast systems: The design and skill assessment of the New York Harbor Observation and Prediction System, version 3 (NYHOPS v3). *Proceedings of the Eleventh International Conference in Estuarine and Coastal Modeling* (pp. 660−685). American Society of Civil Engineering, Seattle, Wash.

Gosavi, A. (2009). Reinforcement learning: A tutorial survey and recent advances. *INFORMS Journal on Computing, 21*(2), 178–192.

Gosavi, A. (2014). *Simulation-based optimization: Parametric optimization techniques and reinforcement learning*. New York, NY: Springer.

Gosavi, A., Das, T. K., & Sarkar, S. (2004). A simulation-based learning automata framework for solving semi-Markov decision problems under long-run average reward. *IIE Transactions, 36*, 557–567.

Granmo, O., Oommen, B., Myrer, S., & Olsen, M. (2007). Learning automata-based solutions to the nonlinear fractional knapsack problem with applications to optimal resource allocation. *IEEE Transactions on Systems, Man and Cybernetics. Part B Cybernetics, 37*(1), 166–175.

Haley, K. (Ed.). (2012). *Search theory and applications*. Berlin: Springer Science & Business Media.

Jacobs, R. A. (1988). Increased rates of convergence through learning rate adaptation. *Neural Networks, 1*(4), 295–307.

Jiang, B., Ravindran, B., & Cho, H. (2013). Probability-based prediction and sleep scheduling for energy-efficient target tracking in sensor networks. *IEEE Transactions on Mobile Computing, 12*(4), 735–747.

Jin, X., Guan, N., Wang, J., & Zeng, P. (2015). Analyzing multimode wireless sensor networks using the network calculus. *Journal of Sensors*, Article ID 851608, 12 pages.

Jung, N.-C., Popescu, I., Kelderman, P., Solomatine, D., & Price, R. (2010). Application of model trees and other machine learning techniques for algal growth prediction in Yongdam reservoir, Republic of Korea. *Journal of Hydroinformatics, 12*(3), 262–274.

Kanevski, M., Timonin, V., & Pozdnukhov, A. (2009). *Machine learning for spatial environmental data: Theory, applications and software*. Lausanne: EPFL Press.

Kwon, J., & Lee, K. M. (2011). Tracking by sampling trackers. *IEEE International Conference on Computer Vision,* 1195–1202.

Le Cadre, H. (2014). Infrastructure topology optimization under competition through cross-entropy. *Journal of the Operational Research Society, 65*(6), 824–841.

Lee, S. (2007). Landslide susceptibility mapping using an artificial neural network in the Gangneung area, Korea. *International Journal of Remote Sensing, 28*(21), 4763–4783.

Li, J., Heap, A. D., Potter, A., & Daniell, J. (2011). Application of machine learning methods to spatial interpolation of environmental variables. *Environmental Modelling and Software, 26*, 1647–1659.

Liang, F., Liu, C., & Carroll, R. (2010). *Advanced Markov chain Monte Carlo methods: Learning from past samples*. Hoboken, NJ: John Wiley & Sons Inc.

Lin, F. Y. S., & Chiu, P. L. (2005). A near-optimal sensor placement algorithm to achieve complete coverage-discrimination in sensor networks. *IEEE Communications Letters, 9*(1), 43–45.

Marjanovic, M., Kovacevic, M., Bajat, B., & Vozenilek, V. (2011). Landslide susceptibility assessment using SVM machine learning algorithm. *Engineering Geology, 123*(3), 225–234.

Martin, R., & Tilak, O. (2012). On ε-optimality of the pursuit learning algorithm. *Journal of Applied Probability, 49*(3), 795–805.

Martinez, S., & Bullo, F. (2006). Optimal sensor placement and motion coordination for target tracking. *Automatica, 42*, 661–668.

Misra, S., Krishna, P. V., & Abraham, K. I. (2011). An adaptive learning scheme for medium access with channel reservation in wireless networks. *Wireless Personal Communications, 56*(1), 55–72.

Misra, S., Tiwari, V., & Obaidat, M. S. (2009). LACAS: Learning automata-based congestion avoidance scheme for healthcare wireless sensor networks. *IEEE JSAC, 27*(4), 466–479.

Narendra, K. S., & Thathachar, M. A. L. (2012). *Learning automata: An introduction*. New York, NY: Dover Publications.

Nicopolitidis, P., Papadimitriou, G., Pomportsis, A., Sarigiannidis, P., & Obaidat, M. S. (2011). Adaptive wireless networks using learning automata. *IEEE Wireless Communications, 18*(2), 75–81.

Nowe, A., Verbeeck, K., & Peeters, M. (2006). Learning automata as a basis for multi-agent reinforcement learning. *Lecture Notes in Computer Science, 3898*, 71–85.

Obaidat, M. S., Papadimitrou, G. I., & Pomportsis, A. S. (2002). Learning automata: Theory, paradigms, and applications. *IEEE Transactions on Systems, Man, and Cybernetics – Part B, 32*(6), 706–709.

Orton, P., Georgas, N., Blumberg, A., & Pullen, J. (2012). Detailed modeling of recent severe storm tides in estuaries of the New York City region, *Journal of Geophysical Research, 117*, C09030. doi: 10.1029/2012JC008220.

Phillips, S. J., Anderson, R. P., & Schapire, R. E. (2006). Maximum entropy modeling of species geographic distributions. *Ecological Modelling, 190*, 231–259.

Pyayt, A. L., Mokhov, I. I., Lang, B., Krzhizhanovskaya, V. V., & Meijer, R. J. (2011). Machine learning methods for environmental monitoring and flood protection, World Academy of Science, Engineering and Technology. *International Journal of Computer, Electrical, Automation, Control and Information Engineering, 5*(6), 549–554.

Ristic, B., Vo, B. N., Clark, D., & Vo, B. T. (2011). A metric for performance evaluation of multi-target tracking algorithms. *IEEE Transactions on Signal Processing, 59*(7), 3452–3457.

Rubinstein, R. Y., & Kroese, D. P. (2013). *The cross-entropy method: A unified approach to combinatorial optimization, Monte-Carlo simulation and machine learning*. Berlin: Springer Science & Business Media.

Simon, H. A. (1959). Theories of decision-making in economics and behavioral science. *The American Economic Review, 49*(3), 253–283.

Simonin, C., Le Cadre, J. P., & Dambreville, F. (2007). The cross-entropy method for solving a variety of hierarchical search problems. *The 10th International Conference on Information Fusion*.

Stone, L. D., Streit, R. L., Corwin, T. L., & Bell, K. L. (2013). *Bayesian multiple target tracking*. Norwood, MA: Artech House.

Thathachar, M. A. L., & Sastry, P. S. (2004). *Networks of learning automata: Techniques for online stochastic optimization*. New York, NY: Springer.

Tien Bui, D., Pradhan, B., Lofman, O., & Revhaug, I. (2012). Landslide susceptibility assessment in Vietnam using Support vector machines, Decision tree and Naıve Bayes models. *Mathematical Problems in Engineering*, Article ID 974638.

Torkestani, J. A., & Meybodi, M. R. (2010). Mobility-based multicast routing algorithm for wireless mobile Ad-hoc networks: A learning automata approach. *Computer Communications, 33*(6), 721–735.

Tsai, D., Flagg, M., Nakazawa, A., & Rehg, J. M. (2012). Motion coherent tracking using multi-label MRF optimization. *International Journal of Computer Vision, 100*(2), 190–202.

Turkogullari, Y. B., Aras, N., Altinel, I. K., & Ersoy, C. (2010). Optimal, placement, scheduling and routing to maximize lifetime in sensor networks. *Journal of the Operational Research Society, 61*, 1000–1012.

Unsal, C., Bay, J. S., & Kachroo, P. (1995). Intelligent control of vehicles: Preliminary results on the application of learning automata techniques to automated highway system. *Proceedings of the 7th International Conference on Tools Artificial Intelligence*, 216–223.

Varshavski, V. I., & Vorontsova, I. P. (1963). On the behavior of stochastic automata with variable structure. *Automat. Telemekh, 24*, 253–360.

Vermeulen, J. F. J., & van den Brink, M. (2005). The search for an alerted moving target. *Journal of the Operational Research Society, 56*, 514–525.

Wallace, J., Tsoularis, A., & Tassabehji, R. (2006). Stochastic automata and supply chain agility in the time-limited supply industry. *International Journal of Agile Systems and Management (IJASM), 1*(4), 407–421.

Wheeler, R., & Narendra, K. S. (1986). Decentralized learning in Finite Markov chains. *IEEE Transactions on Automatic Control, 31*(6), 373–376.

Zeng, X., & Liu, Z. (2005). A learning automata based algorithm for optimization of continuous complex functions. *Information Sciences, 174*, 165–175.

Zhu, Q., Han, Z., & Basar, T. (2010). No-regret learning in collaborative spectrum sensing with malicious nodes. *Proceedings of the IEEE International Conference on Communications (ICC '10)*, Cape Town, South Africa.